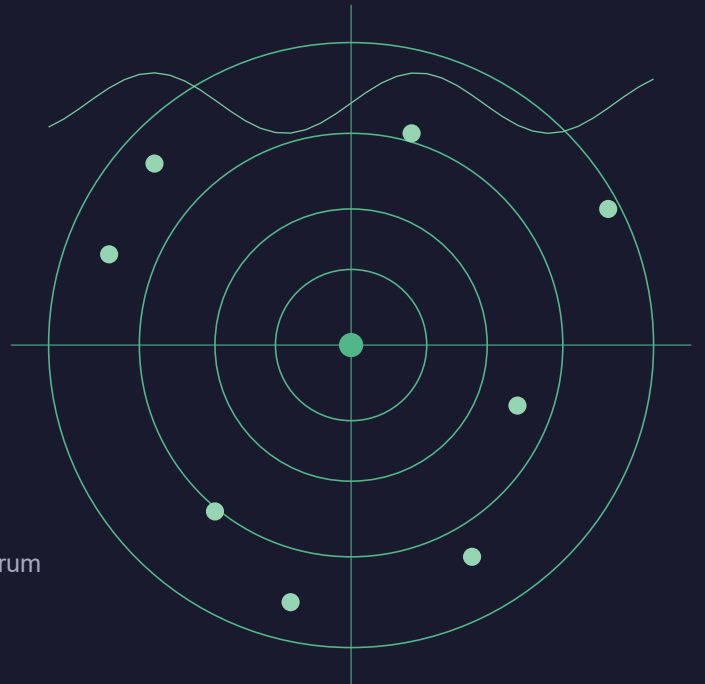


# Steam Yorumları Üzerinde Uçtan Uca Büyük Veri Pipeline'ı ve Duygu Analizi

*Teknik Rapor*



# Teknik Rapor

## Steam Yorumları Üzerinde Uçtan Uca Büyük Veri Pipeline'ı ve Duygu Analizi

Ders: Büyük Veri Analizine Giriş

Dönem: 2025–2026 Bahar

Veri Seti: Steam Reviews (Kaggle, ~21M yorum)

### 1. Proje Mimarisi

Bu projede Kaggle'dan alınan Steam oyun yorumları veri seti üzerinde, endüstriyel standartlara uygun bir uçtan uca büyük veri pipeline'ı kurulmuştur. Pipeline; veri üretimi, akış işleme, depolama, makine öğrenmesi ve görselleştirme olmak üzere beş katmandan oluşmaktadır.

#### 1.1 Katman Yapısı

Konteynerizasyon. Tüm servisler Docker Compose ile yönetilmektedir. `docker-compose.yml` dosyasında Zookeeper, Kafka, Producer, Spark ve Dashboard olmak üzere beş servis tanımlanmıştır. Servisler arası iletişim Docker iç ağı üzerinden sağlanmış; veri dosyaları ve çıktılar host makineyle volume bağlantısıyla paylaşılmıştır.

Veri Üretimi (Kafka Producer). `producer/producer.py`, `steam_reviews.csv` dosyasından (2,06 GB, 5.000.000 satır) satır satır okuyarak her kaydı `event_id`, `event_type`, `timestamp`, `app_id`, `app_name`, `review_text`, `review_score` ve `review_votes` alanlarını içeren JSON formatında `steam_reviews` Kafka topic'ine iletmektedir. `MESSAGE_DELAY_SECONDS=0.0` ve `MAX_MESSAGES=5.000.000` parametreleriyle tam veri seti aktarılmıştır.

Akış İşleme (Spark Structured Streaming). `spark/jobs/stream_steam_reviews_to_delta.py`, Kafka topic'ini sürekli dinleyerek gelen JSON mesajlarını iki Delta Lake katmanına yazmaktadır:

- Bronze Layer (`data/delta/bronze/steam_reviews_raw`): Ham Kafka kayıtları, herhangi bir dönüşüm uygulanmadan saklanmaktadır.
- Silver Layer (`data/delta/silver/steam_reviews_clean`): `review_text` ve `app_name` alanlarında null değer filtresi uygulanmış, `review_score` değerinden sentiment etiketi türetilmiş (1 → "positive", -1 → "negative") ve `event_time` timestamp'i eklenmiş temiz veri saklanmaktadır.

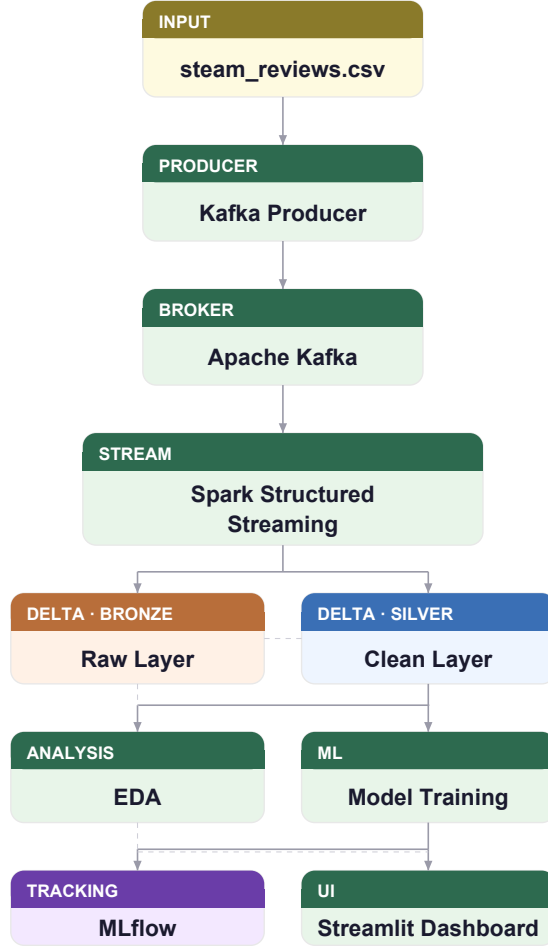
Gold katmanı bu projede uygulanmamıştır; özellik mühendisliği aşamaları doğrudan eğitim script'lerine gömülmüştür.

Makine Öğrenmesi. İki ayrı PySpark MLlib script'i kullanılmıştır. `train_sentiment_model.py` Logistic Regression modelini sınıf başına 100.000 örnekle eğitirken, `train_compare_models_mlflow.py` dört ek modeli (Decision Tree, Random Forest, GBT, Naive Bayes) sınıf başına 25.000 örnekle karşılaştırmıştır. Tüm deneyler MLflow file-store (`reports/mlruns/`) üzerinde takip edilmiştir.

Görselleştirme. Docker servisi olarak çalışan Streamlit tabanlı çok sayfalı bir dashboard (`dashboard/`) oluşturulmuş ve `http://localhost:8501` adresinden erişilebilir hale getirilmiştir.

#### 1.2 Veri Akışı

Aşağıdaki diyagram, ham CSV dosyasından Streamlit dashboard'una kadar tüm akışı ve katmanlar arası ilişkileri özetler. Düz oklar zorunlu veri akışını, kesikli oklar ikincil/destekleyici beslemeleri göstermektedir.



Şekil 1 — Uçtan uca veri akışı: CSV → Kafka → Spark → Delta Lake → EDA / ML → MLflow & Dashboard.

## 2. Keşifsel Veri Analizi Bulguları

Silver Delta katmanındaki 5.000.000 satır üzerinde gerçekleştirilen EDA temel bulguları şunlardır:

- Sınıf Dengesizliği: Veri setinin %81'i pozitif (4.067.557 yorum), %19'u negatif (932.443 yorum) yorumdan oluşmaktadır. Bu dengesizlik, model eğitiminde her sınıftan 25.000 örnek alınarak (limit-based undersampling) giderilmiştir.
- Yorum Uzunluğu: Negatif yorumlar ortalama 388 karakter, pozitif yorumlar ortalama 291 karakterdir. Negatif yorumların ortalama 97 karakter daha uzun olması, mutsuz kullanıcıların daha ayrıntılı geri bildirim verme eğilimini ortaya koymaktadır.
- Oyun Dağılımı: 5.580 benzersiz oyun bulunmakta olup PAYDAY 2 (88.973) ve DayZ (88.850) en fazla yorumlanan oyunlardır.
- Eksik Değer: Silver katmanındaki null filtresi sayesinde hiçbir kolonda eksik değer kalmamıştır.

## 3. Özellik Mühendisliği

Tüm modeller için ortak bir TF-IDF tabanlı metin işleme pipeline'ı kullanılmıştır:



Şekil 2 — Ortak metin işleme pipeline'ı (TF-IDF tabanlı).

Logistic Regression modeli ise daha geniş bir kelime dağarcığını yakalamak amacıyla CountVectorizer yerine HashingTF(numFeatures=20000) kullanmıştır. Bu pipeline review\_text sütunundan 5.000 boyutlu seyrek TF-IDF vektörü üretmektedir. Yalnızca metin tabanlı özellikler kullanılmıştır; review\_text\_length, review\_votes ve app\_review\_count gibi sayısal özellikler modele dahil edilmemiştir.

## 4. Model Sonuçları ve Değerlendirme

|                                          |                                             |
|------------------------------------------|---------------------------------------------|
| <b>EN İYİ MODEL</b><br><br><b>LogReg</b> | <b>AUC-ROC</b><br><br><b>0.9424</b>         |
| <b>ACCURACY</b><br><br><b>0.8795</b>     | <b>TOPLAM YORUM</b><br><br><b>5.000.000</b> |

Beş sınıflandırma modeli eğitilmiş, AUC-ROC başta olmak üzere beş değerlendirme metriği hesaplanmıştır:

| Model                      | Accuracy | F1-Score | Precision | Recall | AUC-ROC         |
|----------------------------|----------|----------|-----------|--------|-----------------|
| <b>Logistic Regression</b> | 0.8795   | 0.8794   | 0.8797    | 0.8795 | <b>0.9424</b> □ |
| Naive Bayes                | 0.8333   | 0.8333   | 0.8334    | 0.8333 | 0.5723          |
| Gradient Boosted Trees     | 0.7341   | 0.7319   | 0.7414    | 0.7341 | 0.8187          |
| Random Forest              | 0.7107   | 0.7029   | 0.7367    | 0.7107 | 0.8014          |
| Decision Tree              | 0.6114   | 0.5691   | 0.6807    | 0.6114 | 0.5157          |

Tablo 1 — Beş model için karşılaştırmalı değerlendirme metrikleri. □ = en yüksek AUC-ROC.

En iyi model Logistic Regression olmuştur. Confusion matrix analizi, TP=16.596, TN=16.921, FP=3.178, FN=3.325 değerlerini ortaya koymuştur. Sensitivity (Recall) 0.833, Specificity 0.842 olarak hesaplanmış; bu değerler modelin iki sınıf arasında dengeli hata yaptığını göstermektedir. ROC eğrisi altındaki alan AUC=0.9424, Youden Index ile belirlenen optimal karar eşiği ise  $t \approx 0.52$ 'dir.

## 5. Karşılaşılan Zorluklar

### 5.1. Model Dosyalarının Bozulması

Windows ortamında Git'in varsayılan CRLF dönüşümü, Spark PipelineModel binary dosyalarını bozmuş ve `org.apache.hadoop.fs.ChecksumException` hatasına yol açmıştır.

*Çözüm: LR modelinin Silver verisinden sıfırdan yeniden eğitilmesiyle çözülmüştür.*

### 5.2. Docker-İçi Yol Sorunu (Path Mangling)

Git Bash ortamında `docker compose run` komutuyla `spark-submit` çalıştırılırken `/opt/spark/bin/spark-submit` yolu Windows formatına dönüştürülmüştür.

*Çözüm: Tüm Docker komutları PowerShell üzerinden çalıştırılarak giderilmiştir.*

### 5.3. Sınıf Dengesizliği

Ham veri setindeki %81/%19 oranındaki dengesizlik model performansını olumsuz etkileyebileceğinden, her sınıftan `limit(N)` kullanılarak eşit sayıda örnek alınmıştır. Bu yaklaşım 5 milyonun üzerinde kaydı devre dışı bırakarak önemli miktarda veri kaybına yol açmıştır.

*Çözüm: Ağırlıklı kayıp fonksiyonu (`weightCol`) veya SMOTE gibi yöntemler daha iyi bir alternatif sunabilirdi.*

### 5.4. Karşılaştırma Adaletsizliği

Logistic Regression modeli diğer dört modelden farklı koşullarda eğitilmiştir: sınıf başına 100.000 örnek ve `HashingTF(20000)` pipeline'ı kullanılmıştır. Diğer modeller sınıf başına 25.000 örnekle ve daha küçük bir kelime dağarcığıyla (`CountVectorizer, 5.000`) eğitilmiştir.

*Çözüm: Bu durum LR'nin yüksek AUC değerinin (0.9424) kısmen eğitim avantajından kaynaklandığını göstermektedir.*

### 5.5. Hiperparametre Optimizasyonu Eksikliği

Hesaplama maliyeti nedeniyle `CrossValidator` veya `TrainValidationSplit` kullanılmamıştır. Tüm modeller varsayılan veya sabit hiperparametrelerle eğitilmiştir (örn. `maxDepth=5, numTrees=20`).

*Çözüm: Sistematik bir grid search ile özellikle GBT ve Random Forest modellerinde belirgin performans iyileştirmesi sağlanabilirdi.*

### 5.6. Ingestion Zamanı Tutarsızlığı

Kafka Producer'ın mesajlara `datetime.now()` ile timestamp ataması, tüm yorumların 2 saatlik bir pencereye sıkışmasına yol açmıştır. Bu durum zaman serisi analizini anlamsız kılmıştır; grafik, yorum gönderim zamanını değil yalnızca pipeline ingestion hızını yansıtmaktadır.

*Çözüm: Üretici tarafında orijinal `timestamp_created` alanının korunması veya sentetik tarih dağılımı ile zenginleştirilmesi gerekirdi.*

## 6. Genel Değerlendirme

Proje, uçtan uca büyük veri pipeline'ının tüm temel bileşenlerini başarıyla hayata geçirmiştir. Kafka aracılığıyla gerçek zamanlı veri akışı simüle edilmiş, Spark Structured Streaming ile 5 milyon satırlık ham veri Bronze ve Silver Delta katmanlarına aktarılmış, beş farklı sınıflandırma modeli MLflow ile karşılaştırmalı olarak takip edilmiş ve Streamlit tabanlı interaktif bir dashboard ile tüm bulgular görselleştirilmiştir.

En iyi model olarak Logistic Regression,  $AUC-ROC=0.9424$  ve  $Accuracy=0.8795$  değerleriyle öne çıkmıştır. Bu sonuç, TF-IDF tabanlı metin özelliklerinin Steam yorumlarında duygu analizi için yeterince ayırt edici olduğunu göstermektedir.

Gelecek çalışmalar kapsamında yorum uzunluğu, helpful votes ve oyun bazında yorum oranı gibi sayısal özelliklerin modele eklenmesi, Gold Delta katmanının oluşturulması, stratified split ve sistematik hiperparametre optimizasyonunun uygulanması düşünülebilir.